

Unified Business Identifier and Active Business Intelligence

A Proposal for Karnataka Commerce & Industries

AI for Bharat 2 — Round 1 Submission

Submission summary. We propose a federated entity-resolution and activity-inference platform that sits alongside Karnataka’s 40+ State department systems without modifying any of them. The platform produces a hierarchical Unified Business Identifier (UBID) anchored to PAN where available, classifies each UBID as Nascent, Active, Dormant or Closed using cadence-aware event evidence, and routes every ambiguous decision through an explainable reviewer workflow whose feedback continuously recalibrates the linkage models.

Contents

1	Problem Understanding	2
1.1	The constraints, read carefully	2
1.2	Granularity: a hierarchy, not a single ID	3
1.3	What success looks like in queries	3
2	Source Schemas and What They Tell Us	3
2.1	The PAN and GSTIN anchors, used carefully	4
3	Part A: Entity Resolution	4
3.1	Stage 1: Canonicalization	4
3.2	Stage 2: Blocking	5
3.3	Stage 3: Pairwise Scoring	6
3.4	Stage 4: Confidence Calibration and Thresholds	6
3.5	Stage 5: Clustering Across Multiple Pairs	7
4	Part B: Activity Inference	7
4.1	Why naive thresholding fails	7
4.2	Per-signal cadence-aware decay	7
4.3	From decayed signals to verdict	8
4.4	Explainability of the verdict	9
4.5	Edge cases the activity engine handles by design	9
4.6	Quarantine for unjoined events	9
5	Human-in-the-Loop Review	9
5.1	Triage: what the reviewer sees	9
5.2	What the reviewer does	10
5.3	Two-tier reviewer structure	10
5.4	Feedback loop: how decisions improve the system	10

6	Architecture	10
6.1	Component overview	11
6.2	Storage choices and why	11
6.3	Model choices and why	12
6.4	Data flow	12
7	Respecting the Non-Negotiables	12
8	Risks, Trade-offs, and How We Handle Them	13
8.1	Technical risks	13
8.2	Operational and political risks	14
8.3	Trade-offs we made deliberately	14
9	Implementation Plan for Round 2	14
10	Edge Cases, Catalogued	15
11	Closing Statement	17

1 Problem Understanding

Karnataka Commerce & Industries cannot today answer first-order questions about its own industrial base — how many businesses are operating, in what sectors, where, and with what recent activity — because the State’s regulatory data is fragmented across 40+ department systems, each built in isolation with its own schema, identifiers, and validation rules. The same business exists as different records in different databases; central anchors like PAN and GSTIN are only partially captured; activity data sits trapped inside each department system. Migrating to a unified database is not realistic at this scale, and the source systems cannot be modified. The platform must therefore be additive: a federation layer that joins what exists rather than replaces it.

We read the problem as two coupled sub-problems with an asymmetric structure.

Part A is an entity resolution problem on dirty, federated master data. Multiple independent systems hold rows describing business entities, none designed to interoperate, and we must determine which rows across systems refer to the same real-world business and assign each business a single identifier.

Part B is a state inference problem on an event stream that is not natively keyed by our identifier. Once UBIDs exist, events arriving from department systems use *their own* record IDs; we must project events onto UBIDs, and from the resulting per-UBID timeline infer a latent operational state.

These problems are sequenced — B is meaningless without A — but they are also *mutually informing*: a UBID that never appears in any event stream is suspicious, and reviewer corrections to A change the evidence base for B.

1.1 The constraints, read carefully

Each non-negotiable in the brief carries operational consequences that shape the architecture.

No source-system changes.

Linkage cannot be a foreign key written back into department databases. The UBID layer must be an external graph that consumes pull-based or stream-subscription ingest. Schema discovery is our problem; canonicalization happens in our layer, never theirs.

Synthetic / scrambled data only in the sandbox.

Our matching cannot rely on memorising real entities. Models must generalise from structural patterns,

which constrains how data-hungry our approach can be. It also rules out hosted-LLM calls and any third-party API that would receive and possibly cache real Karnataka business addresses.

Every decision explainable and reversible.

Pure black-box embedding-based matching cannot be the deciding layer. Embeddings (or any opaque feature) may contribute as one signal among many, but the auto-link decision must be decomposable into evidence a reviewer can read. Reversibility means UBIDs are mutable assignments, not destructive merges — architecturally, we maintain a versioned linkage table on top of source records.

Wrong merge > missed merge in cost.

False positives compound: once two distinct businesses share a UBID, all their events co-mingle and the verdict for both is corrupted. This is a precision-over-recall preference and pushes auto-link thresholds conservative, with the cost of a longer reviewer queue.

1.2 Granularity: a hierarchy, not a single ID

A central design question is what a UBID actually identifies. Three reasonable definitions exist: a legal entity (one PAN, one company), an establishment (a specific premise), or an operating unit (an establishment plus its workforce / license footprint). Different department systems naturally key on different definitions: GST registers legal entities (state-scoped); Factories Act registers establishments; KSPCB consents are per-premise; Shop & Establishment registrations are per-branch. Picking one wrong definition breaks the whole system.

Design choice

We adopt a **three-layer hierarchy**: a Legal Entity layer anchored to PAN; a UBID layer at the establishment level (one premise, one operational unit, one UBID); and a Source Records layer linking each department's row to the UBID. Connection-level identifiers (BESCOM RR numbers, water connections) attach as a fourth, infrastructure-keyed layer because they often do not carry PAN and must be matched through address.

This mirrors how the source data is actually organised. It handles the proprietorship case correctly — a single PAN with multiple unrelated establishments stays as separate UBIDs under one Legal Entity — and lets us anchor multi-state operations cleanly: a Mumbai HQ with a Bangalore branch shares a Legal Entity node, but only the Bangalore establishment receives a Karnataka UBID.

1.3 What success looks like in queries

The platform succeeds when Karnataka Commerce & Industries can run a query like “*active factories in pin code 560058 with no inspection in the last 18 months*” against a single UBID-keyed warehouse and receive an answer with the supporting evidence. Today this query is impossible because (i) “factories” is locked inside the FBIS database, (ii) “inspection” events are scattered across multiple sources, and (iii) there is no join key to bring them together. Our deliverable is the join key, the evidence, and the verdict.

2 Source Schemas and What They Tell Us

Every design decision downstream depends on what each source system actually exposes. We examined the public documentation of four representative Karnataka systems — e-Karmika (Shop & Establishment), FBIS (Factories), KSPCB XGN (pollution consents), and BESCOM (electricity) — plus the structure of PAN and GSTIN as central anchors. The findings dictate our blocking, matching, and activity strategies.

Three observations from this table drive our design.

First, KSPCB is the cleanest source for addresses because it stores Industrial Area, Taluk, and District as separate fields. We use KSPCB's structured-address vocabulary as the seed dictionary for our locality normaliser — a free, government-validated source for canonical Bengaluru/Karnataka locality

Source	Identifier(s) it exposes	PAN / GSTIN coverage	Address quality
e-Karmika (S&E)	Establishment registration number	PAN partial, GSTIN rare	Free text; Kannada + English
FBIS (Factories)	License number (Form 3); Form 2 reg. no.	PAN at user signup; GSTIN inconsistent	Mixed; structured at registration, drift on amendment
KSPCB XGN	Consent file number	PAN typically present	Best in class — separate fields for Industrial Area, Taluk, District
BESCOM	RR Number, Account ID, Consumer/K Number	None — legacy connection records	Free text; consumer name often a person, not the business

Table 1: Identifier and address-quality summary across four representative sources.

names.

Second, BESCOM is the messiest source for entity resolution but the richest for activity inference. Its identifiers are inconsistent even internally (the connection-holder name on a meter is frequently the property owner, not the business), and PAN is absent. But it generates monthly bills, payments, and consumption readings — a high-frequency continuity signal unavailable elsewhere. We treat BESCOM as a separate sub-resolution problem: cluster its connections among themselves first, then attach the cluster to a UBID via address-based matching with conservative thresholds.

Third, regime changes break naive time-series interpretation. The 2019 Karnataka S&E amendment abolished periodic renewal in favour of self-certification, so a renewal filing pre-2019 is a strong activity signal whereas post-2019 it is near-automatic. The 2024 KSPCB consolidation of Combined Consent and Authorization (CCA) orders changed the document structure of consents. Our event ingestion must be regime-aware: each event type carries an effective-date window, and decay parameters are scoped to that window.

2.1 The PAN and GSTIN anchors, used carefully

PAN is a 10-character code where the 4th character encodes entity type — P for individual, C for company, F for partnership firm, H for HUF, T for trust, and so on. This is a free, structural signal: when we see a PAN with 4th character P, the holder is a natural person, and we must *not* collapse multiple establishments owned by that person into one business — they share a Legal Entity parent (the proprietor) but remain distinct UBIDs.

GSTIN is a 15-character code structured as [state:2] [PAN:10] [entity:1] [Z] [checksum:1]. The state code is national but *the embedded PAN is the same across states*, so we extract digits 3–12 to recover the legal-entity anchor for cross-state operations. The 13th digit lets a single legal entity hold multiple GSTINs in the same state (different business verticals), which means GSTIN-to-UBID mapping is many-to-one, not one-to-one. Our schema models this explicitly.

3 Part A: Entity Resolution

The goal of Part A is to produce, for each real-world business establishment, a single UBID that links its records across source systems — with confidence-tagged evidence, conservative auto-merging, and a reviewer queue for ambiguity. Our approach is a four-stage pipeline: *canonicalize*, *block*, *score*, *cluster*, with explainability built into every stage.

3.1 Stage 1: Canonicalization

Each source system feeds an *adapter* that projects its records into a common internal schema. Adapters perform deterministic, source-specific cleaning without touching the original data. The canonical record

carries: source identifier, original record ID (preserved verbatim for traceability), normalised name, normalised address with extracted components, structural identifiers (PAN, GSTIN, derived legal-entity PAN from GSTIN), sector category, contact fields, and ingestion metadata.

Name normalisation. A deterministic pipeline strips legal-form noise (“Pvt Ltd”, “M/s”, “Private Limited”, “LLP”, “& Sons”, “& Co”), expands common abbreviations (“Trdrs” → “Traders”), normalises case and whitespace, and produces a token list. Kannada-script names are transliterated using *indic-transliteration* (open source, runs locally) into a Roman canonical form so that “*Sharma Traders*” and its Kannada equivalent block together. We deliberately do *not* use neural embeddings for name matching: Bengaluru-specific business vocabulary (locality names, sector slang, family-name compounds) is poorly represented in pretrained multilingual models, the marginal lift is small, and the explainability cost is high. Probabilistic string distances (Jaro-Winkler on tokens, token-set ratio, Jaccard on character n-grams) give us decomposable, defensible features.

Address canonicalization. This is a first-class subsystem, not a post-processing step. The pipeline:

1. **Pin code extraction** via regex. Pin code is the strongest stable signal in an Indian address and partitions the search space cleanly. It is the primary blocking key.
2. **Component parsing** via a hand-built grammar tuned for Bengaluru patterns: door numbers, cross/main numbering (with ordinal ↔ Roman conversion), stage / phase / block markers.
3. **Locality normalisation** via a curated synonym dictionary. “Peenya 2nd Stage”, “Peenya II Stage”, “Peenya Industrial Area Phase 2” all collapse to a canonical key. The dictionary is bootstrapped from KSPCB’s structured Industrial-Area / Taluk / District fields, then extended via reviewer feedback.
4. **Optional geocoding** for the lat/lng signal. We propose self-hosted Nominatim (OpenStreetMap) for the production path; quality is patchy in India but the data never leaves our environment. As a sandbox-only enrichment, an India-resident geocoder may be considered.

Design choice

We deliberately rule out hosted address-validation services (e.g. Google Address Validation API) for the production path, despite their convenience. Three reasons: (i) it ships State PII to a third-party cloud, conflicting with the data-handling non-negotiable; (ii) Indian coverage is in pre-GA preview with no version-compatibility guarantee; and (iii) standard Maps Platform terms restrict long-term storage of canonicalized output, which is incompatible with a persistent UBID layer. An in-house deterministic parser plus a curated locality dictionary is more work upfront but is the only architecturally defensible choice for a State-government data system.

3.2 Stage 2: Blocking

All-pairs comparison across millions of records is computationally intractable ($\sim 10^{12}$ comparisons). Blocking reduces this to candidate pairs without losing true matches. We use *multiple blocking keys with union*, so a pair is a candidate if it shares any key:

- **PAN equality** (when both records have PAN) — highest precision block.
- **Derived legal-entity PAN equality** (extracted from GSTIN) — catches multi-vertical and multi-state cases.
- **Pin code + first 4 chars of normalised name (Soundex)** — the workhorse block.
- **Pin code + door number prefix** — catches name-mismatch but same-premise cases.
- **Phone number** — secondary, since phones are reused across businesses.

Blocking is implemented as inverted indices in OpenSearch. The union-of-blocks approach trades a slightly larger candidate set for substantially better recall, which matters because recall lost at blocking is recall lost forever.

3.3 Stage 3: Pairwise Scoring

For each candidate pair we compute a feature vector and a calibrated match probability. The scoring layer is two-tiered: a **deterministic tier** that auto-decides high-confidence cases by rule, and a **probabilistic tier** for everything else.

Deterministic tier. If two records share a valid, non-null PAN *and* their other strong fields (pin code, normalised name top-2 tokens) are not in active conflict, we accept the pair as a deterministic match (probability ≈ 1). Symmetrically, if two records have different non-null PANs, we reject the pair as a deterministic non-match. PAN equality with one record having PAN 4th-char P (proprietorship) is downgraded to *soft* match: same legal owner, possibly different establishments, route to address comparison.

These deterministic rules cover an estimated 30–50% of cases at high precision and bypass the model entirely. They are the easiest to explain (“these two records share PAN ABCDE1234F and pin code 560058”).

Probabilistic tier. For the remaining pairs we compute a feature vector with ~ 25 hand-crafted features:

- **Name features:** Jaro-Winkler on full normalised name; token-set ratio; Jaccard on character 3-grams; longest common substring length; legal-form-stripped exact match indicator.
- **Address features:** pin code equality; door-number equality; locality canonical-key match; cross/main numeric distance; Levenshtein on residual address tokens; geographic distance (when geocoded).
- **Identifier features:** GSTIN equality; phone equality; email domain match; partial-PAN agreement (for cases where one side has PAN and the other has GSTIN).
- **Structural features:** sector-category compatibility; size-class compatibility (employee count, sanctioned load); registration-date proximity.

Design choice

We use **LightGBM** for pairwise scoring, calibrated post-hoc with isotonic regression. Reasons: (i) gradient-boosted trees handle our mix of continuous, binary, and categorical features natively; (ii) per-feature SHAP values give us the explainability the brief demands — every match carries “these features pushed the score up, these pushed it down”; (iii) LightGBM is fast at inference, which matters at scale; (iv) it trains well on small labelled sets, which we will have in the synthetic-data sandbox. We considered Fellegi-Sunter probabilistic record linkage (the classical approach). It is more interpretable in its raw form but makes strong feature-independence assumptions that break for our correlated features (name and address signals are not independent). LightGBM gives us better calibration and recall at the cost of slightly more training infrastructure — a trade we accept.

3.4 Stage 4: Confidence Calibration and Thresholds

A raw model score is not a probability. A score of 0.85 must *actually* mean that 85% of pairs scored 0.85 are true matches; otherwise our auto-link / review / reject thresholds are arbitrary. We calibrate using **isotonic regression** on a held-out validation set, which is non-parametric and handles the non-linear miscalibration typical of boosted trees. We monitor calibration drift via reliability diagrams refreshed weekly.

Threshold setting. Given the calibrated probability p and the brief’s directive that wrong merges cost more than missed merges, we set:

- $p \geq 0.95$: **auto-link**. At this threshold, expected precision is high enough that the false-merge cost is acceptable; we further require that no deterministic-tier rule rejects the pair.
- $0.55 \leq p < 0.95$: **review queue**. Routed to a human reviewer with the feature breakdown.
- $p < 0.55$: **reject (keep separate)**. Logged for audit but not surfaced.

These thresholds are not hard-coded constants; they are tuned on the synthetic ground truth in Round 2. The auto-link threshold is the single most important operating parameter, and we expose it as a runtime configuration so that Karnataka C&I can tighten or loosen it based on reviewer capacity.

3.5 Stage 5: Clustering Across Multiple Pairs

Pairwise scores produce edges in a graph. Naïve transitive closure is dangerous — if A-B and B-C are both 0.95 but A-C is 0.40, blindly merging {A,B,C} would be wrong. We use **correlation clustering** with a positive/negative edge formulation: edges with $p \geq 0.95$ are positive, $p < 0.20$ are negative, the middle range is unobserved. The clustering objective minimises disagreements (positive edges across clusters plus negative edges within clusters).

Design choice

For tractability at scale we use a greedy correlation-clustering heuristic with conflict-aware merging: candidate merges are evaluated against existing cluster members and rejected if they would violate a strong negative edge. This preserves the precision-over-recall preference and produces auditable merge histories. For the Round 2 sandbox we additionally run `networkx.connected_components` on positive-only edges as a sanity baseline.

Reversibility. Cluster assignments are versioned. Every merge carries a timestamp, a contributing-edge set, and a reviewer-id (or auto). A reviewer’s must-link or cannot-link decision becomes a constraint that downstream clustering passes must respect. We never destructively rewrite source identifiers — the UBID is a layer on top, queryable via a lookup service.

4 Part B: Activity Inference

Once UBIDs exist, events arrive from department systems keyed by their own record IDs. We project each event onto a UBID via the linkage table; events that cannot be confidently joined go to a quarantine queue (Section 4.6). The remaining UBID-keyed event timeline drives the activity verdict.

4.1 Why naive thresholding fails

The simplest approach — “any event in the last 90 days → Active” — breaks for several reasons. Different events have different *activity strength* (an electricity payment is a continuity signal; a state-driven inspection is not a business signal at all). Different events have different *natural cadences* (annual returns are once a year; meter readings are monthly; not seeing each for 6 months means very different things). And dormancy is a *real distinct state*, not a halfway score: a fireworks shop is genuinely dormant for nine months and active for three.

4.2 Per-signal cadence-aware decay

Each event type T has an expected cadence τ_T and a decay function $d_T(\Delta t)$. The contribution of an event of type T that occurred Δt days ago is:

$$c_T(\Delta t) = w_T \cdot \exp\left(-\frac{\Delta t}{\alpha \cdot \tau_T}\right)$$

where w_T is the per-signal weight (continuity strength) and α controls how forgiving the decay is relative to expected cadence. A factory not filing Form 20 ($\tau = 365$ days) is forgiven for 6 months; a BESCOM connection generating no bill ($\tau = 30$ days) is not.

Source	Event	Strength w_T	Cadence τ_T	Sign	Notes
S&E (pre-2019)	Renewal filed	0.7	1825d (5y)	+	Strong signal, periodic
S&E (post-2019)	Self-cert + fee	0.3	365d	+	Weak — near-automatic
S&E	Closure intimation	1.0	—	terminal	Deterministic Closed
Factories	Form 20 (annual return)	0.8	365d	+	Due Feb 1
Factories	Form 21 (half-yearly)	0.7	183d	+	Due Jul 15
Factories	License renewal	0.9	varies	+	1–3y cycle
Factories	Inspection / accident	0.4	—	ambiguous	State-driven, not business-driven
KSPCB	CFO re-newal	0.9	varies	+	Per category (R/O/G)
KSPCB	Compliance report	0.6	180d	+	Periodic per category
BESCOM	Bill generated	0.5	30d	+	Monthly
BESCOM	Bill paid (non-zero)	0.9	30d	+	Strongest continuity signal
BESCOM	Zero-consumption month	0.4	30d	—	Negative evidence
BESCOM	Disconnect	0.9	—	terminal-ish	Strong but reversible

Table 2: Catalogue of activity signals. Weights and cadences are starting values; reviewer corrections in Part A and verdict-correction labels in Part B drive supervised re-tuning.

4.3 From decayed signals to verdict

For each UBID we aggregate decayed contributions into a continuity score $S = \sum_T \sum_{e \in \text{events}(T)} c_T(\Delta t_e)$, plus a vector of per-signal aggregates that preserve which sources contributed. The verdict is produced in three stages:

1. **Deterministic overrides first.** If a formal closure event exists (S&E closure intimation, KSPCB consent revoked, factory delicensed), the verdict is **Closed** regardless of score. If only a registration event exists and the UBID is less than 90 days old, the verdict is **Nascent**.
2. **Rule-bounded thresholding.** The continuity score is mapped through learned thresholds tuned on the synthetic ground truth. Default cuts: $S \geq 1.5 \rightarrow \text{Active}$, $0.4 \leq S < 1.5 \rightarrow \text{Dormant}$, $S < 0.4 \rightarrow \text{Closed-by-silence}$.
3. **Sector-aware adjustment.** Seasonal sectors (event halls, fireworks, agricultural processing) carry a sector prior that widens the Dormant band and slows the Closed transition. The prior is loaded from a sector configuration file, not learned — it must be auditable.

Design choice

We deliberately keep the activity engine **rule-driven with tuned parameters**, not end-to-end learned. A learned classifier (HMM, sequence model) would be more powerful but harder to explain and harder to debug when a verdict surprises a department. A rule engine where every parameter has a name and a justification is what survives a procurement review. The tuning process can be supervised — reviewers correcting verdicts produce labels — but the model remains a transparent function.

4.4 Explainability of the verdict

Every verdict is shipped with an *evidence timeline*: the events that contributed, their decayed weights, the deterministic rules that fired, and the threshold band. A reviewer (or department officer) sees: “*Closed because no electricity bill paid in 14 months, no Form 20 filed in 2024, KSPCB consent expired 2023-08, no overriding renewal event detected.*” The same data is exposed via API so that a department’s own dashboards can surface it.

4.5 Edge cases the activity engine handles by design

- **Seasonal businesses.** Sector prior widens Dormant band. A wedding-hall UBID with summer-only events stays Dormant in winter, not Closed.
- **Late filings burst.** Three years of returns filed in one month boost the score, but each event’s Δt is computed from its filing date, not its period date, so a true catch-up is correctly weighted.
- **Bill paid by absent owner.** Single signal, low marginal contribution; if BESCOM payments are the *only* activity, the UBID stays Dormant rather than auto-promoting to Active.
- **Migration to a new establishment.** If A and B are correctly linked as one UBID (Part A), events from both contribute. If they are mistakenly separate, A goes Closed-by-silence and B becomes Nascent — a pattern the reviewer queue surfaces for re-linkage (Section 5).
- **Brand-new UBID with one event.** Nascent state, hold-out period before any Closed transition.

4.6 Quarantine for unjoined events

Events arrive keyed by source-system identifiers. Most join cleanly to a UBID via the linkage table. Some do not — the source row has not been ingested yet, the linkage was below auto-link threshold, or the event refers to a sub-identifier (BESCOM RR number) not yet attached. These events go to a quarantine queue, indexed by source identifier, and re-played whenever the linkage table is updated. We *never silently drop* an unjoined event; the reviewer dashboard shows quarantine size as a top-level metric.

5 Human-in-the-Loop Review

The reviewer workflow is the system’s correctness backstop. It exists for two reasons: to handle ambiguous matches the model cannot decide, and to feed labelled corrections back into the model. Both are first-class.

5.1 Triage: what the reviewer sees

The review queue is not a flat list. It is sorted by *informativeness* — cases where reviewer effort produces the most learning. Informativeness combines:

- **Uncertainty:** pairs whose calibrated probability sits near 0.7–0.85 (high disagreement) are prioritised over pairs near 0.55 (likely no-match).
- **Impact:** clusters whose merge would affect many downstream events get higher priority.
- **Disagreement among signals:** a pair where name match is high but address match is low is more informative than a pair where everything weakly agrees.

This is active learning by another name. It keeps reviewer throughput producing maximal model improvement.

5.2 What the reviewer does

The console presents a candidate pair (or candidate cluster) with:

- Side-by-side records with raw and canonicalized fields highlighted.
- Per-feature contribution from the model (SHAP-style “name pushed +0.18, address pushed −0.12”).
- Hits from the deterministic tier, if any.
- Existing must-link / cannot-link constraints affecting the cluster.

The reviewer decides: **Confirm match**, **Reject (keep separate)**, **Defer to senior**, or **Flag data quality issue** (e.g. source record looks corrupt). Decisions are timestamped and reviewer-attributed.

5.3 Two-tier reviewer structure

We propose a tiered structure to balance throughput and quality: **junior reviewers** for routine ambiguity (5–10 min per case), **senior reviewers** for escalations and policy-setting cases (cross-state mergers, owner-change disputes). A junior reviewer can defer to senior; a senior reviewer’s decision is binding and becomes a precedent (added to the must-link / cannot-link constraint set).

5.4 Feedback loop: how decisions improve the system

1. **Direct labels.** Each reviewer decision is a labelled pair. Labels accumulate in a training table.
2. **Periodic retraining.** The pairwise model is retrained on a schedule (weekly initially, monthly at steady state) and re-calibrated. New thresholds are A/B-tested against the previous model on a held-out set before promotion.
3. **Constraint propagation.** Must-link and cannot-link decisions are stored as graph constraints and respected by every subsequent clustering pass. A reviewer’s “these are different businesses” decision survives even if the model’s score on the pair drifts upward.
4. **Synonym dictionary updates.** When a reviewer marks a match where two locality strings refer to the same place, the canonical-locality dictionary gets a new entry, improving *future* blocking and matching.
5. **Activity-engine corrections.** A reviewer who flags an Active/Dormant/Closed verdict as wrong produces a label that re-tunes the activity thresholds and per-signal weights.

6 Architecture

The platform sits as a federation layer between source department systems and Karnataka C&I’s analytical workloads. Source systems remain untouched. We pull or stream-subscribe to their data; nothing flows the other way.

6.1 Component overview

Component	Role
Source adapters	One per department system. Convert source-specific schema into the canonical record. Pull-based ingest where possible; stream-subscription where supported.
Canonicalization service	Name normalisation, address parsing, identifier extraction. Stateless; deterministic.
Blocking index (OpenSearch)	Inverted indices on PAN, GSTIN, derived-PAN, pin+name-soundex, pin+door, phone.
Pairwise scorer (LightGBM service)	Featurisation + calibrated probability. Two-tier (deterministic + probabilistic).
Clustering service	Greedy correlation clustering with constraints from reviewer feedback. Maintains versioned cluster history.
UBID graph (Neo4j)	The entity hierarchy: Legal Entity → UBID → Source Records → Connection IDs. Versioned merges and constraint edges.
Linkage lookup (Redis + Postgres)	Hot-path map from any source identifier → UBID. Postgres is source of truth; Redis is read cache.
Event ingestion	Kafka topics per source. Events are joined to UBIDs via linkage lookup; misses go to quarantine.
Event store (DuckDB on Parquet)	Append-only UBID-keyed event log. Partitioned by month and source.
Activity engine	Periodic batch (hourly) + streaming updates. Computes continuity score and verdict per UBID.
Reviewer console	Web UI (FastAPI + thin React frontend). Active-learning queue, side-by-side comparison, decision capture.
Query API	Read-side service exposing UBID lookup, status, event timeline, and the exemplar query class (“active factories in pin X with no inspection in 18m”).

6.2 Storage choices and why

Postgres for the canonicalized record store and the linkage table. ACID, mature, supports versioned writes via temporal tables, and is the source of truth.

OpenSearch for blocking indices and fuzzy text search in the reviewer console. Inverted indices make union-blocking fast; n-gram analysers handle approximate matching natively.

Neo4j for the entity graph. The hierarchy (Legal Entity → UBID → Source Records) plus must-link / cannot-link constraints is fundamentally graph-shaped. Relational joins to traverse this would be painful; Cypher queries are natural.

DuckDB on Parquet for the event warehouse. Append-only, columnar, fast for the analytical query class. No need for a heavyweight warehouse at expected event volumes; DuckDB embeds in the analytics service and reads Parquet directly from object storage.

Redis for the linkage hot-path. UBID lookups are read-heavy and latency-sensitive; Redis caches the source-id → UBID map with TTL-based invalidation on linkage updates.

Kafka for event ingestion. Decouples source-adapter cadence from consumer cadence; quarantine is a topic, not a special data path.

This stack is heavier than strictly necessary for an MVP. A simpler “Postgres for everything plus a Python service” would ship faster but hits walls at scale: blocking becomes slow, graph queries become painful, and event analytics blocks transactional reads. We choose the more complete stack because the platform’s value compounds with scale and a serious procurement process will demand the components anyway. The Round 2 prototype can use SQLite/DuckDB substitutes for the blocking and graph layers to demonstrate the design without requiring full infrastructure — the interfaces stay the same.

6.3 Model choices and why

Pairwise scorer: *LightGBM with isotonic calibration.*

Justified in Section 3. The single most important learned component.

Activity engine: *rule-driven with tuned parameters.*

Justified in Section 4. We deliberately do not use a sequence model here.

Name matching: *probabilistic string distances, no embeddings.*

Bengaluru-specific business vocabulary (locality names, sector terms, family-name compounds) is poorly represented in pretrained multilingual models. The lift from embeddings is small, training is unstable on small synthetic data, and the explainability cost is high. We commit to Jaro-Winkler, token-set ratio, and character n-gram Jaccard as the name-comparison primitives.

Address normalisation: *deterministic parser plus curated dictionary.*

Avoids hosted APIs (data-residency), avoids neural parsing (explainability), and is bootstrappable from KSPCB’s clean structured fields.

Transliteration: *indic-transliteration (open source, local).*

Kannada ↔ Roman conversion needs to be deterministic and on-prem.

6.4 Data flow

A new source-system snapshot arrives. The adapter projects it to the canonical schema. The canonicalization service normalises names and addresses. New / changed records are indexed in OpenSearch. The pairwise scorer fetches candidate pairs by union-block, computes features, and emits scored pairs. The clustering service consumes scored pairs, applies constraints, and updates the UBID graph in Neo4j; the linkage table in Postgres is updated atomically; Redis cache is invalidated for affected source identifiers. Any pair in the review band emits a record to the reviewer queue. Reviewer decisions write back to the constraint set and the labelled-pair table. Independently, event topics fan in from source systems; events are joined to UBIDs through Redis (Postgres on miss); joined events append to the DuckDB store; unjoined events queue for replay. The activity engine reads the event store on a rolling window and produces verdicts that materialise into a per-UBID status table queried by the API.

7 Respecting the Non-Negotiables

Each constraint from the brief shapes a concrete part of the system. We enumerate them with the corresponding architectural responses.

No source-system changes. The platform is strictly downstream. Source-system adapters consume snapshots, change-data-capture streams, or scheduled exports — whichever each system supports today. No adapter writes back to a source system. The UBID is held in our graph; source records keep their original identifiers, and we map between the two via the linkage table. If a source system adds a new field tomorrow, only its adapter changes; the canonical schema absorbs the difference.

Synthetic / scrambled data only. Every learned component (pairwise scorer, isotonic calibrator, activity-engine parameter tuning) is trained and validated on synthetic ground truth in the Round 2 sandbox. We do not bake real-data assumptions into models. The deterministic tier (PAN/GSTIN equality rules, address grammar) is structural, not data-dependent, and works equally on real or scrambled inputs. We make zero hosted-LLM calls, and we do not send data to third-party APIs in the production path; address validation is in-house, transliteration is in-house, geocoding (where used) is self-hosted Nominatim.

Explainable and reversible decisions. Every linkage carries a versioned evidence record: which features contributed, which deterministic rules fired, which reviewer (or auto) made the call. The UBID graph stores merges as discrete events with full lineage; an unmerge is a first-class operation that propagates to the linkage table, the event-quarantine queue, and downstream verdicts. Activity verdicts are produced by a transparent rule chain with named parameters; the verdict API returns the evidence timeline alongside the label.

Wrong merge > missed merge. The auto-link threshold is conservative ($p \geq 0.95$). The deterministic non-match rule (different non-null PANs) is a hard reject. Clustering is correlation-clustering with conflict resolution rather than transitive closure, so weak chains do not collapse distinct entities. Reviewer-issued cannot-link constraints are durable: the model can never auto-merge a pair that a reviewer has rejected.

8 Risks, Trade-offs, and How We Handle Them

8.1 Technical risks

Calibration drift.

As source data evolves, the relationship between model score and true match probability can drift. *Mitigation:* reliability diagrams refreshed weekly, automated alerts when calibration error exceeds a threshold, scheduled recalibration on the latest reviewer-labelled set.

Address normalisation gaps.

Our locality dictionary will miss long-tail areas, especially outside Bengaluru. *Mitigation:* reviewer feedback captures every locality-pair the system gets wrong, dictionary grows monotonically. Initial coverage focused on Bengaluru Urban (the brief's sample area) and expanded as new districts are ingested.

BESCOM data quality.

Connection-holder names are often individuals, not businesses; internal identifier inconsistency is documented. *Mitigation:* BESCOM connections cluster among themselves first (RR $\leftrightarrow$ Account ID $\leftrightarrow$ Consumer ID), then attach to a UBID via address only, with the auto-link threshold raised. BESCOM-only UBIDs (no S&E / Factories / KSPCB linkage) are flagged for reviewer attention before being committed.

Quarantine queue growth.

If a large source system is delayed, unjoined events pile up. *Mitigation:* quarantine size is a top-level dashboard metric; replay is automatic on linkage updates; aged quarantine items (> 90 days) generate alerts.

Reviewer bottleneck.

If auto-link threshold is too conservative, reviewer queue grows unboundedly. *Mitigation:* threshold is a runtime parameter; queue length is monitored; an active-learning ordering keeps marginal reviewer effort high-value even under load.

8.2 Operational and political risks

Source-system access friction.

Departments may resist exposing data, even read-only. *Mitigation:* ingestion design is flexible (SFTP drops, snapshot APIs, message queues); legal/data-sharing agreement happens before adapter implementation; pilot with the 3–4 most cooperative departments first.

Schema drift in source systems.

A department changes its schema without notice. *Mitigation:* adapters validate canonical-schema conformance on every batch and fail loudly rather than silently corrupting data. Alerts go to the platform team and the source-system owner.

Definitional disputes.

Two departments may disagree about whether a closed business was Dormant or Closed at a given point. *Mitigation:* the activity engine produces *evidence*, not just a label. The label is auditable and overrideable; departments retain authority over their own data.

8.3 Trade-offs we made deliberately

Trade-off

Precision over recall in linkage. We set $p \geq 0.95$ for auto-link and route the 0.55–0.95 band to humans. This is a deliberate cost-sensitive choice (wrong merges > missed merges) at the price of a larger reviewer workload. We do not regret this.

Trade-off

Rule-driven activity engine over a learned sequence model. A learned model would likely score higher on a benchmark; a rule engine survives a procurement review. We chose the rule engine.

Trade-off

No neural embeddings in the matching path. We forgo the marginal lift in name-similarity recall in exchange for explainability and stability under synthetic data. If, in Round 2, the LightGBM scorer's recall on the genuine ambiguous tail falls short, we revisit this — but only with a self-hosted Indic model and only as one feature, never as the deciding layer.

Trade-off

Heavier storage stack vs. MVP simplicity. A Postgres-only MVP would ship faster but caps the system at small scale. We choose the complete stack with substitutes (DuckDB, NetworkX in-memory, Postgres-only blocking) acceptable for the Round 2 prototype.

9 Implementation Plan for Round 2

Assuming a sandbox with representative synthetic data and APIs is provided, we propose a phased implementation. Phases are stated as logical milestones rather than calendar weeks; each milestone gates the next.

Phase 1 — Canonical schema and adapters

Implement the canonical record schema. Build adapters for the 3–4 source systems supplied in the sandbox (anticipated: e-Karmika, FBIS, KSPCB, BESCOB equivalents). Ship deterministic name and address normalisation. Verify canonicalization output by sampling and visually inspecting a few hundred records per source.

Phase 2 — Blocking and deterministic tier

Stand up OpenSearch (or a Postgres-trigram substitute for the prototype). Implement union-blocking. Build the deterministic match tier (PAN equality, GSTIN-derived PAN, hard non-match rule). Measure: candidate-pair volume, deterministic-tier coverage, false-positive rate on the synthetic ground truth.

Phase 3 — Probabilistic scorer and clustering

Featurisation, LightGBM training on synthetic labels, isotonic calibration. Greedy correlation clustering with constraints. Wire up the UBID graph (NetworkX in-memory for the prototype, Neo4j-shaped). Measure: pairwise precision/recall, cluster-level B³ precision/recall, calibration error.

Phase 4 — Reviewer console

FastAPI + thin frontend. Active-learning queue ordering. Side-by-side record comparison with feature contributions surfaced. Decision capture and constraint storage. Run a small simulated review cycle on the synthetic data.

Phase 5 — Event ingestion and activity engine

Event topics, joining via the linkage map, quarantine for misses. The signal catalogue with starting weights and cadences. Decay computation, deterministic overrides, threshold-band classifier, sector priors. Verdict API with evidence timeline.

Phase 6 — Query layer and exemplar queries

The exemplar query class (*active factories in pin X with no inspection in N months*) implemented end-to-end against the UBID-keyed warehouse. A small dashboard exposing UBID count, cluster-size distribution, queue length, quarantine size, and verdict distribution by sector.

Phase 7 — Hardening and feedback loop

End-to-end run on the full synthetic dataset. Reviewer-correction loop demonstrated — a labelled correction visibly changes a subsequent verdict. Calibration monitoring. Documentation of every parameter, threshold, and dictionary entry.

Stretch

- Self-hosted Nominatim for geocoding, contributing a distance feature to the scorer.
- A second adapter beyond the four primaries (BWSSB or a fire-safety system) to demonstrate adapter pluggability.
- A reversibility demo: a reviewer un-merges a previously-auto-merged pair and the system propagates the reversal through the event store and verdict layer.

What we will measure

The submission will report, on the synthetic ground truth: pairwise precision and recall at the auto-link threshold; cluster-level B³ precision/recall; calibration error (Brier score, ECE); fraction of records auto-linked vs. reviewed vs. rejected; activity-verdict accuracy by class (Nascent, Active, Dormant, Closed); state-transition latency on simulated regime-change scenarios; reviewer-loop demonstration (a corrected label visibly changes future scoring).

10 Edge Cases, Catalogued

For completeness, the table below enumerates known edge cases and the system's planned response. Many are addressed inline in earlier sections; this is the audit-friendly reference.

Edge case	System response
Address change. Business moves premises.	New UBID for the new establishment; reviewer can issue a must-link to the old UBID if continuity is established. The Legal Entity parent stays the same.
Ownership change / acquisition. Same premises, new PAN.	New Legal Entity parent, same UBID for the establishment; a successor edge in the graph captures continuity.
Sublet premises. Two unrelated businesses share an address.	Pin + door-number block surfaces both; pairwise scorer rejects on name + sector mismatch despite address match. Distinct UBIDs.
Mall / industrial estate. Many businesses, one master address.	Address-only matching is blocked from auto-link in known multi-tenant zones; reviewer required for first-time entries.
Family-run cluster. Same surname, same address, separate firms.	Distinct PANs → distinct UBIDs under deterministic non-match. If both share a proprietor's PAN, soft-match and route to reviewer.
Phantom entities. Records that look real but never had an event.	Stay Nascent until aged out, then flagged for reviewer scrutiny.
Multi-state legal entity. One PAN, GSTINs in multiple states.	Legal Entity node holds all GSTINs; Karnataka UBIDs only attach to the Karnataka establishment.
PAN reused across linked entities. Proprietorship, multiple firms.	PAN 4th-char P downgrades the match to soft; address must agree to merge.
Pre-PAN-mandate records. No PAN, no GSTIN.	Soft-match path only; auto-link threshold raised.
Kannada-script duplicates. Same business in different scripts.	Transliteration normalisation collapses scripts before blocking.
Deliberate aliasing. Slightly different names across departments.	Token-set ratio and address agreement compensate; reviewer arbitrates surviving ambiguity.
GSTIN cancelled / suspended. Formatted but inactive.	Active-status check at ingest where possible; cancelled GSTIN is down-weighted as a match feature; treated as a (weak) Closed signal for activity.
Late filings burst. Years of returns in one month.	Δt computed from filing date, not period date; per-event decay handles correctly.
Bill paid by absent owner. BESCOM payments only, nothing else.	Single signal → score insufficient for Active; remains Dormant.
Power disconnect. Owner dispute, temporary.	Disconnect is strong but not terminal; reversible on subsequent reconnect event.
Migration to new establishment. Old UBID silent, new UBID active.	Pattern surfaces in reviewer queue (paired Closed-by-silence and Nascent in same locality) for re-linkage.
Brand-new UBID, registration only.	Nascent until first activity event, hold-out period before any Closed transition.
Schema drift in source. Field added or renamed.	Adapter validates canonical-schema conformance and fails loudly.
Source delay. Department snapshot late.	Events queue in quarantine; replay automatic on snapshot arrival.
Reviewer disagreement. Two reviewers reach different verdicts on similar cases.	Senior tier sets precedent via must-link / cannot-link; precedents are versioned and auditable.
Conflicting sector classifications. S&E says “hotel”, KSPCB says “manufacturing”.	Sector is a multi-valued feature with provenance, not a single field; activity prior uses the most-specific classification.

Edge case	System response
Closure not reported. Business stops filing without intimation.	Closed-by-silence after sufficient signal decay; reversible if activity resumes.

11 Closing Statement

What we are proposing is not exotic. The components are well-understood — federated ingestion, probabilistic record linkage, calibrated classification, rule-driven state inference, active-learning review — and the architectural choices favour boring correctness over novelty. The hard work in this problem is not invention; it is the disciplined application of the right primitives to a domain whose data is messy, whose constraints are real, and whose stakes (a wrong merge corrupts every event for two businesses) are unforgiving.

We have made every design choice with three things in mind: that the platform must sit alongside source systems without disturbing them, that every decision must be readable by a human, and that wrong merges are worse than missed merges. The architecture, the model choices, the threshold settings, and the reviewer workflow all fall out of these three commitments.

Karnataka Commerce & Industries should, when this platform is operating, be able to ask its industrial base questions that are impossible today — and receive answers backed by evidence it can verify. That is the deliverable.